

Cours Python : Introduction à Xarray

Pourquoi Xarray

- Xarray est une bibliothèque Python pour la manipulation de données multi-dimensionnelles.
- Inspirée par **Pandas**, elle facilite l'analyse de données N-dimensionnelles.
- Elle est idéale pour travailler avec des données météorologiques indexées selon **lat, lon, time**.
- Xarray gère différents formats de données utilisés en météorologie (**NetCDF**, grib, zarr) => Permet d'avoir le même code applicatif prêt à l'ouverture du fichier.
- **Gère et propage** les métadonnées tout au long des calculs.

TP

Durant ce TP, nous allons voir comment ouvrir et manipuler quelque peu un fichier NetCDF (de données AROME) avec xarray.

Nous verrons :

- Comment avoir de l'information sur les variables
- Comment sélectionner des sous-plages
- Comment faire des calculs
- Comment faire des plots

Pour ce TP il vous faudra installer `xarray` et `netCDF4` .

- Activer l'environnement virtuel
- `pip install xarray netCDF4`

Ouverture d'un fichier NetCDF

```
import xarray as xr
# Permet de conserver les attributs après les opérations
xr.set_options(keep_attrs=True)
# Ouverture du fichier
ds = xr.open_dataset("/home/newton/horse/destouchesm/COURS_CS/arome_forecast_2024100900.nc")
#Affiche le contenu du fichier
print(ds)
```

Résultat de l'ouverture

```
<xarray.Dataset> Size: 69MB
Dimensions:          (latitude: 1200, longitude: 1600, time: 4)
Coordinates:
  * latitude          (latitude) float64 10kB 53.4 53.39 53.38 ... 41.42 41.41
  * longitude         (longitude) float64 13kB -8.0 -7.99 -7.98 ... 7.98 7.99
  * time              (time) datetime64[ns] 32B 2024-10-09T01:00:00 ... 2024...
    heightAboveGround float64 8B ...
Data variables:
    altitude          (latitude, longitude) float32 8MB ...
    t2m               (time, latitude, longitude) float32 31MB ...
    r2                (time, latitude, longitude) float32 31MB ...
Attributes:
    GRIB_edition:      2
    GRIB_centre:       1fpw
    GRIB_centreDescription: French Weather Service - Toulouse
    GRIB_subCentre:    0
    Conventions:       CF-1.7
    institution:       French Weather Service - Toulouse
    history:            2024-10-10T09:28 GRIB to CDM+CF via cfgrib-0.9.1...
```

Qu'avons nous chargé ?

Un fichier contenant trois variables :

- **t2m** (temperature)
- **r2** (humidité spécifique)
- **altitude**

Ces variables possèdent des coordonnées (**time, latitude, longitude**)

Le fichier, ainsi que l'ensemble des variables et coordonnées, possède des attributs.

Ex : `history` renseigne sur la manière dont a été créé le fichier.

Dans le *vocabulaire* xarray, une variable particulière d'un *Dataset* s'appelle un *dataArray*.

Exercice 1

En consultant les attributs de la variable `t2m`, retrouver en quelle unité est stockée la température.

Sélection de données

Il y a deux manières principales de sélectionner des données :

- par valeur

```
t2m_toulouse = ds["t2m"].sel(latitude=43.6, longitude=1.43, method="nearest")
```

`method=nearest` permet de sélectionner le point de grille le plus proche.

- par indice

```
t2m_by_index = ds["t2m"].isel(latitude=198, longitude=705)
```

Sélection de données : par plage

On peut aussi sélectionner via une liste ou des "slices"

```
t2m_area = ds["t2m"].sel(latitude=slice(44, 42), longitude=slice(0, 2))
```

Attention : La sélection suivante n'aurait sélectionné aucune donnée !

```
t2m_area = ds["t2m"].sel(latitude=slice(42, 44), longitude=slice(0, 2))
```

En effet, les latitudes sont rangées de manière décroissante dans ce fichier.

Exercice 2

1. Retrouver la position (en latitude/longitude) du point de grille situé à l'index (198,705) de l'exemple précédent.
2. Quelle est la température à 2h du matin à Paris ?
3. Combien de points de grille ont été sélectionnés par la commande suivante ?

```
t2m_area = ds["t2m"].sel(latitude=slice(44, 42), longitude=slice(0, 2))
```

Calculs sur les données

On peut directement faire des calculs sur un `Dataset` ou sur un `DataArray`

```
# Moyenne temporelle pour toutes les variables du dataset
ds_temporal_mean = ds.mean(dim="time")
# Variation spatiale
ds_std = ds.std(dim=["latitude", "longitude"])
```

NB On peut aussi employer directement des fonctions `numpy` sur un `DataArray`.

```
np.mean(ds["t2m"], axis=0)
# is equivalent to
ds["t2m"].mean(dim="time")
```

Exercice 3

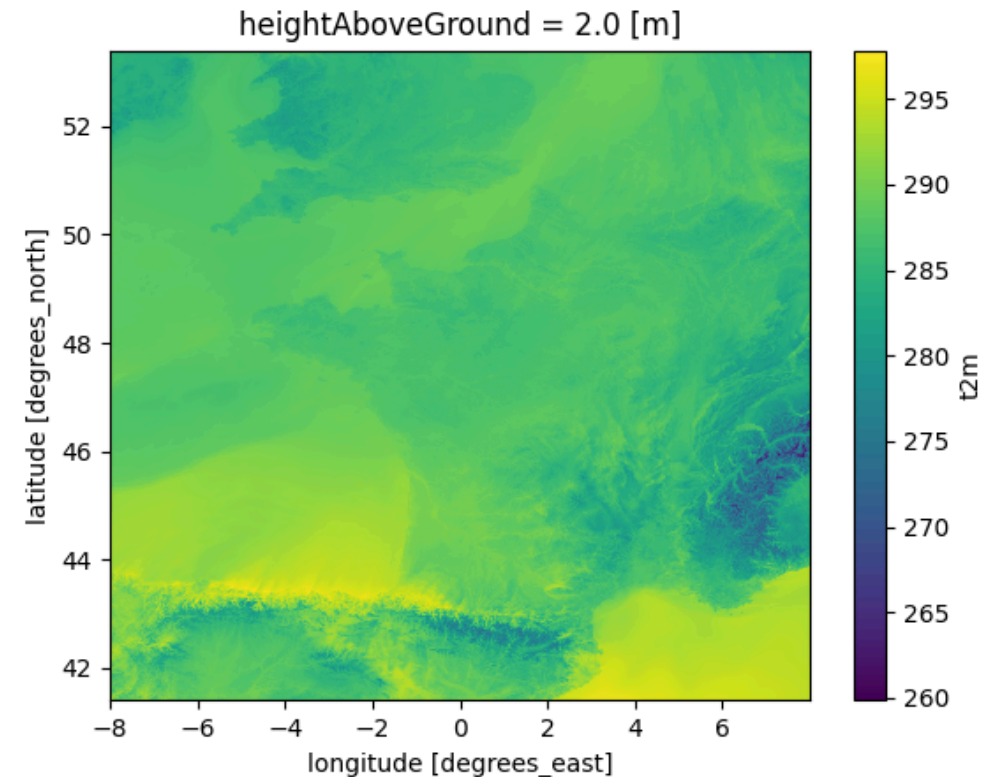
- Quelles sont les dimensions de vos variable après la moyenne temporelle ?
- Quelle est la température moyenne sur la période sur un rectangle
 - latitude : [42, 44]
 - longitude : [0, 2]

Visualisation rapide (via matplotlib)

Attention : On peut uniquement utiliser la fonction `plot` sur les **DataArray**.

```
import matplotlib.pyplot as plt
ds_temporal_mean["t2m"].plot()
plt.show()
```

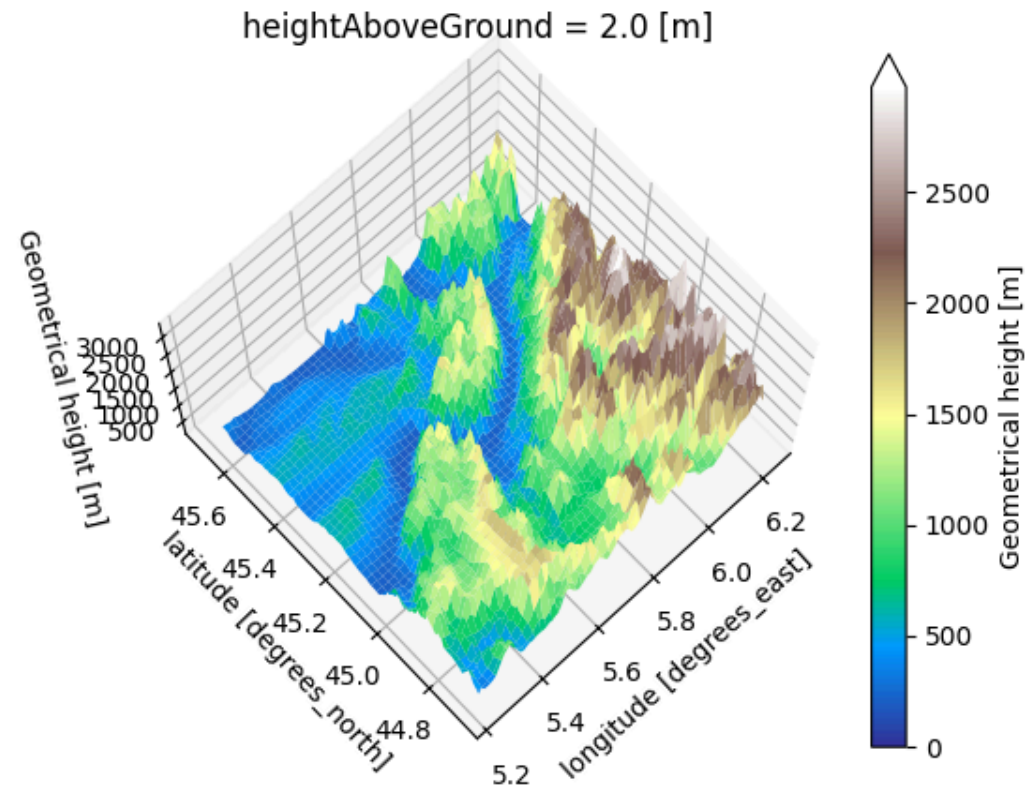
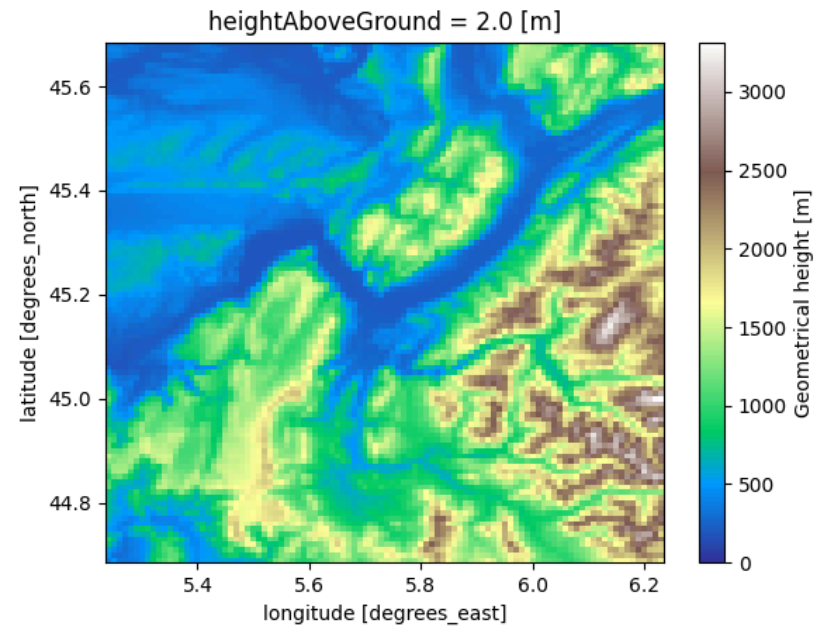
Bonus : On a directement accès aux coordonnées sur le graphique.



Exercice 4

- Afficher l'altitude sur un carré d'un degré de côté centré autour de Grenoble.

Solution 4 : rendu



Utilisation de masques pour calculer sur des sous-domaines

Avec `xarray` (et `numpy`) On peut aisément créer des masques et les utiliser. Cela peut se faire comme dans l'exemple suivant avec `where` ou par simple multiplication.

```
mask = ds["altitude"] > 1000
temperature_above_1000 = ds["t2m"].where(mask)
mean = temperature_above_1000.mean()
print(f" Température moyenne des points ayant une altitude supérieure à 1000 m : {str(mean)} {mean.units}")
```

```
> Temperature moyenne des points ayant une altitude supérieure à 1000 m : 281.15 K
```

Exercice 5

Faites une fonction permettant de calculer la température moyenne d'une zone donnée pour les points situés au dessus d'une certaine altitude.

Exercices - A rendre

1. Améliorer la fonction de l'exercice 5 de telle sorte à prendre en entrée la variable d'intérêt dans le dataset (par ex. `r2` ou `t2m`)
2. Combien y a-t-il d'occurrences de température négative (en °C) dans le dataset entier ? Quel pourcentage de cas cela représente-t-il ?
3. Calculer la moyenne glissante (sur 3h) pour la zone autour de Grenoble. (Indication : utiliser la fonctionnalité `rolling` de *xarray*)

Le dossier `/home/newton/horse/destouchesm/COURS_CS/data` contient un fichier `grid_arpege.nc` .

Ce fichier contient une unique variable `g1ob0125` donnant une information d'altitude sur le domaine AROME pour ARPEGE (modèle global). La résolution de l'orographie est ici de 0.125° (~12.5 km), résolution bien plus lâche que celle d'AROME (~1.3km).

Exercices - A rendre

4.a. Faire une figure montrant l'orographie des deux modèles (AROME et ARPEGE).
Qu'observe-t-on ? Que pouvez vous dire de l'altitude maximum sur chacune de ces cartes ?

4.b Interpoler l'orographie d'ARPEGE sur celle d'AROME (vous pouvez utiliser `interp_like` de xarray).

Quelle est le maximum de différence (en valeur absolue) entre les deux orographies ?

4.c Même question mais en interpolant l'orographie d'AROME sur l'orographie d'ARPEGE.